

CampusEats — Design

Assignment 2: Services, Contracts & Schema

Student: Sandeep Gupta

Submission document: Tasks 1, 3, 4 and 6

Task 1 — Capability List

A capability is a distinct job or responsibility performed by the CampusEats system. The six capabilities below are taken from the CampusEats system description in the course material.

#	Capability	Description
1	Manage Profile & Addresses	Allows students to manage their profile and delivery addresses.
2	Browse Restaurants & Menus	Allows students to browse campus restaurants, menus, and prices.
3	Add to Cart & Place Order	Allows students to add food items to a cart and place an order.
4	Pay / Refund	Handles online payments and refunds for orders.
5	Assign Rider & Track Delivery	Assigns a rider for delivery and supports delivery tracking.
6	Send Notifications	Sends notifications as the order moves through stages such as placed, paid, on the way, and delivered.

Conclusion: CampusEats has six distinct capabilities: manage profile & addresses; browse restaurants & menus; add to cart & place order; pay / refund; assign rider & track delivery; and send notifications.

Service Ownership and Responsible Member

Service	Owns	Member
Accounts	users, addresses, login	Sandeep Gupta
Catalogue	restaurants, menus, prices	Sandeep Gupta
Orders	carts, orders, order status	Sandeep Gupta
Payments	transactions, refunds	Sandeep Gupta
Delivery	riders, assignments	Sandeep Gupta
Notifications	message log	Sandeep Gupta

Task 3 — Service Contracts

A service contract states what other services may call: operation name, input, output, and errors. It does not expose database tables, ID formats, or SQL.

Orders Service

Operation	Input	Output	Errors
placeOrder	studentId; items [{itemId, qty}]; deliveryAddress; paymentMethodId; estimatedMinutes	orderId; status; items; total; deliveryAddress	empty cart; item unavailable; invalid address; payment method not found
getOrder	orderId	orderId; status; items; total; deliveryAddress	order not found
cancelOrder	orderId	orderId; status	order not found; order cannot be cancelled

Catalogue Service

Operation	Input	Output	Errors
listRestaurants	area / filter	restaurants[]	none
getMenu	restaurantId	menu[]	restaurant not found
checkItem	itemId; quantity	itemId; available; price	item not found; item unavailable

Payments Service

Operation	Input	Output	Errors
charge	orderId; amount; paymentMethodId	transactionId; status; receipt	payment declined; invalid payment method

Delivery Service

Operation	Input	Output	Errors
assignRider	orderId; deliveryAddressId	assignmentId; riderId; status	no rider available; invalid delivery address

Notifications Service

Operation	Input	Output	Errors
send	userId; message; channel	notificationId; status	invalid user; delivery failed

Accounts Service

Operation	Input	Output	Errors
getProfile	userId	userId; name; email; addresses	user not found

Task 4 — Full Specification: placeOrder

Central operation: placeOrder. It is the Orders operation that accepts the student's order details, validates the request, coordinates with other service contracts, and returns an order confirmation.

1. Input

- **studentId**
- **items:** [{itemId, qty}]
- **deliveryAddressId**
- **paymentMethodId**

2. Output

- **orderId**
- **status**
- **total**
- **estimatedMinutes**

3. Error Cases

Error	Meaning
Empty cart	No items were supplied for the order.
Item unavailable	One or more requested items are unavailable.
Invalid address	The supplied delivery address is not valid.
Payment declined	The payment cannot be completed using the supplied payment method.

4. Internal Details Hidden from Callers

The public contract does not expose internal storage, identifier generation, total-calculation logic, item-availability implementation, payment-provider implementation, rider-selection logic, delivery-estimation logic, database tables, or SQL statements. Callers depend only on the published operation contract.

5. Service Interactions

During order placement, Orders coordinates through published operations:

1. Orders → Catalogue: **checkItem**
2. Orders → Payments: **charge**
3. Orders → Delivery: **assignRider**
4. Orders → Notifications: **send**

6. Successful Flow

Student sends order details → Orders validates the request → Catalogue checks requested items → Payments processes the payment → Delivery assigns a rider → Notifications sends an order notification → Orders returns the order confirmation.

Task 6 — Five Properties of a Service

The CampusEats service design is checked against the five service properties from the course material.

Service	Reachable	Self-contained	Contract	Independent	Loosely coupled
Accounts	PASS	PASS	PASS	PASS	PASS
Catalogue	PASS	PASS	PASS	PASS	PASS
Orders	PASS	PASS	PASS	PASS	PASS
Payments	PASS	PASS	PASS	PASS	PASS
Delivery	PASS	PASS	PASS	PASS	PASS
Notifications	PASS	PASS	PASS	PASS	PASS

1. Reachable Over a Network — PASS

Other services call published operations such as `checkItem`, `charge`, `assignRider`, and `send`.

2. Self-contained — PASS

Each service owns and manages its own data slice: Accounts owns users/addresses/login; Catalogue owns restaurants/menus/prices; Orders owns carts/orders/status; Payments owns transactions/refunds; Delivery owns riders/assignments; Notifications owns the message log.

3. Has a Contract — PASS

Each service exposes operations with defined inputs, outputs, and errors. Internal implementation is hidden.

4. Independent — PASS

Each service has a clear responsibility and can change its internal implementation without requiring callers to know its internal database or code.

5. Loosely Coupled — PASS

Services communicate through published contracts instead of directly accessing another service's internal database tables.

Conclusion: The designed CampusEats services satisfy the five required service properties. The boundaries are based on capabilities and data ownership, services expose contracts, and cross-service communication uses operations rather than direct access to another service's internal data.